

- Вводное

Результаты для параметров, которые были указаны в примере в репозитории

Обучение: Model: LogisticRegression penalty: l2 C: 0.9 solver: lbfgs max_iter: 1000	Обработка данных: params: features: - race - sex - native.country - occupation - education - capital.gain train_size: 1000
--	--

Полные параметры:

Parameter	Value
l1_ratio	0.0
max_iter	1000
n_jobs	None
penalty	l2
random_state	42
solver	lbfgs
tol	0.0001
verbose	0
warm_start	False
train_size	1000
features	race,sex,native.country,occupation,education,capital.gain
run_hash	260b1efcce59
model_type	LogisticRegression
C	0.9
class_weight	None
dual	False
fit_intercept	True
intercept_scaling	1

Результат:

Metric	Value
accuracy	0.7953731190500563
f1	0.32941965783294197
roc_auc	0.719667323597866
precision	0.765990639625585
recall	0.20982905982905983
average_precision	0.5150996541909055

Добавим в обучение все признаки. Нам не нужна интерпретация коэффицентов модели, тем более, будем использовать не только линейную. Потому, больше данных = больше скор на тесте, гипотетически

Обучение: Model: LogisticRegression penalty: l2 C: 0.9 solver: lbfgs max_iter: 1000	Обработка данных: params: age,workclass,fnlwgt,education.num,marital. status,relationship,capital.loss,hours.per.wee k,race,sex,native.country,occupation,educati on,capital.gain (все 14/14 признаков) train_size: 1000
--	--

Полные параметры:

Parameter	Value
train_size	1000
features	age,workclass,fnlwgt,education.num,marital.status,relationship,capital.loss,hours.per.week,race,sex,native.country,occupation,education,capital.gain
run_hash	fa5058d2ddc0
model_type	LogisticRegression
C	0.9
class_weight	None
dual	False
fit_intercept	True
intercept_scaling	1
l1_ratio	0.0
max_iter	1000
n_jobs	None
penalty	l2
random_state	42
solver	lbfgs
tol	0.0001
verbose	0
warm_start	False

Результат:

Metric	Value
accuracy	0.8102159893540792
f1	0.5410891089108911
roc_auc	0.8094426669335809
precision	0.6429411764705882
recall	0.4670940170940171
average_precision	0.6318846171884809

В Т3 указана метрика ROC-AUC как ориентировочная. ROC_AUC_все признаки = 0.81, ROC_AUC_ограниченные = 0.72. Значительное улучшение. По остальным метрикам тоже виден прогресс.

- Эксперимент 1

Гипотеза: Если увеличить количество тренировочных данных, то качество станет лучше.

Возьмем модель

Model: LogisticRegression

penalty: l2

C: 0.9

solver: lbfgs

max_iter: 1000

Со всеми (14/14 фичами)

И будем перемещаться по сетке train_size [1000, 10000] с шагом в 1000 + запуск на всех данных (22792)

Comparing 11 Runs from 1 Experiment

Visualizations

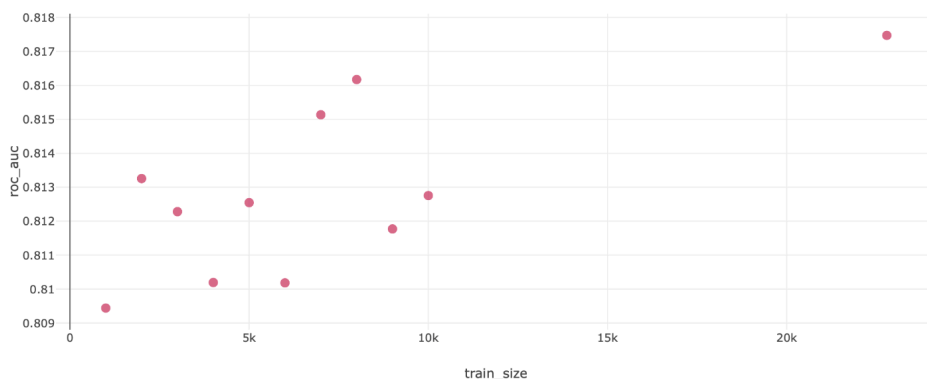
Parallel Coordinates Plot **Scatter Plot** Box Plot Contour Plot

X-axis:

train_size

Y-axis:

roc_auc



Хотелось бы получить более явную линейную зависимость

Comparing 11 Runs from 1 Experiment

Visualizations

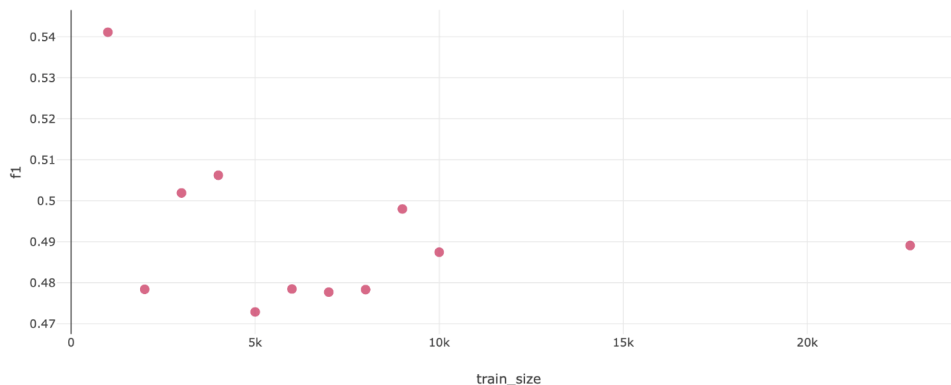
Parallel Coordinates Plot Scatter Plot Box Plot Contour Plot

X-axis:

train_size

Y-axis:

f1



По F1 что то вообще странное

Comparing 11 Runs from 1 Experiment

Visualizations

Parallel Coordinates Plot Scatter Plot Box Plot Contour Plot

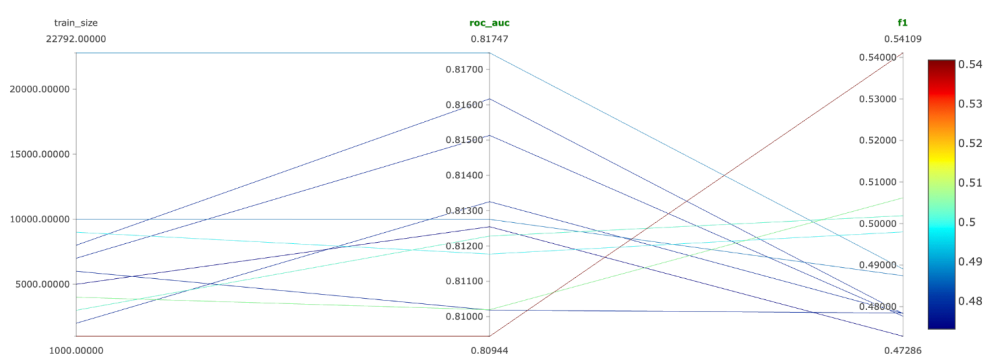
Parameters:

train_size X

Metrics:

roc_auc X f1 X

Clear All



Но видно, что при train_size = всему размеру датасета по целевой метрике roc_auc получилось больше всего значение = 0.81747. Минимальное значение метрики у train_size = 1000 = 0.80944. Далее так и будем использовать весь датасет (22792) для обучения.

- Эксперимент 2

Гипотеза: Ансамблевые алгоритмы обычно проявляют себя лучше, чем линейные модели. А также увеличение кол-ва оценщиков в ансамбле увеличивает качество.

Посмотрим на Random Forest с n_estimators = 100, 300, 500, 700, 1000.

Comparing 5 Runs from 1 Experiment

Visualizations

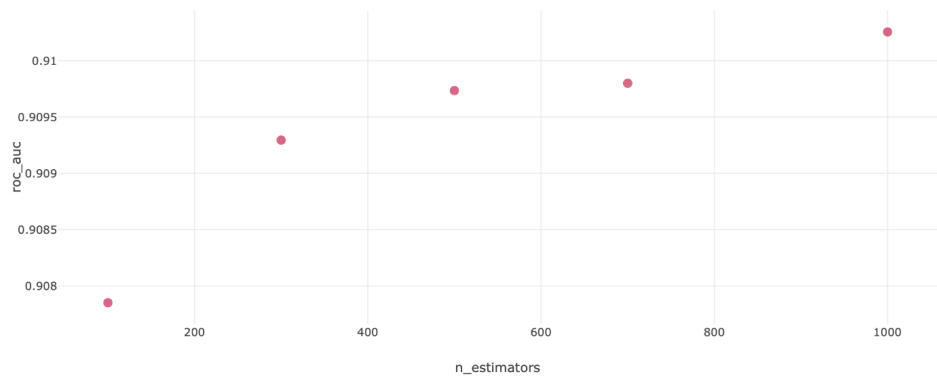
Parallel Coordinates Plot Scatter Plot Box Plot Contour Plot

X-axis:

n_estimators

Y-axis:

roc_auc



Comparing 5 Runs from 1 Experiment

Visualizations

Parallel Coordinates Plot Scatter Plot Box Plot Contour Plot

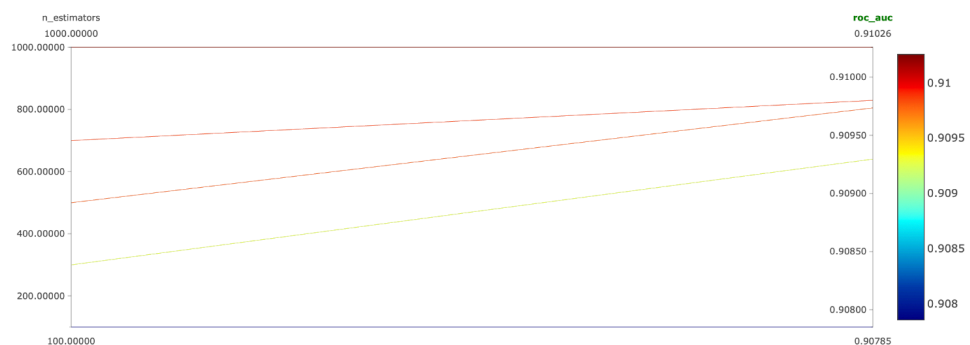
Parameters:

n_estimators X

Metrics:

roc_auc X

Clear All



Да, действительно, увеличение количества деревьев положительно влияет на метрику, хоть абсолютная разница и небольшая (0.90785 - 0.91026).

Сразу видно, как с применением другого типа модели roc_auc возрастает с максимального на логреге = 0.8174 до минимального на случайном лесу = 0.90785.

- Эксперимент 3

Гипотеза: А что, если ансамбля не надо и хватит одного решающего дерева?

Протестируем 1 решающее дерево с разной max_depth = 3, 5, 7, 10 и None (ограничение не накладывается)

Comparing 5 Runs from 1 Experiment

Visualizations

Parallel Coordinates Plot Scatter Plot Box Plot Contour Plot

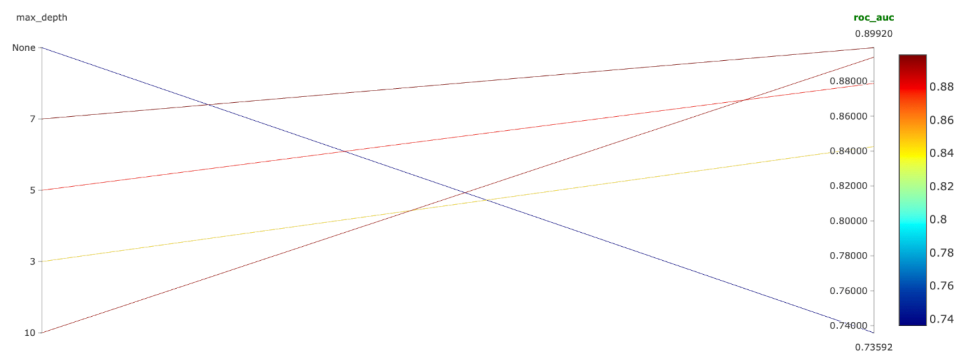
Parameters:

max_depth X

Metrics:

roc_auc X

Clear All



Comparing 5 Runs from 1 Experiment

Visualizations

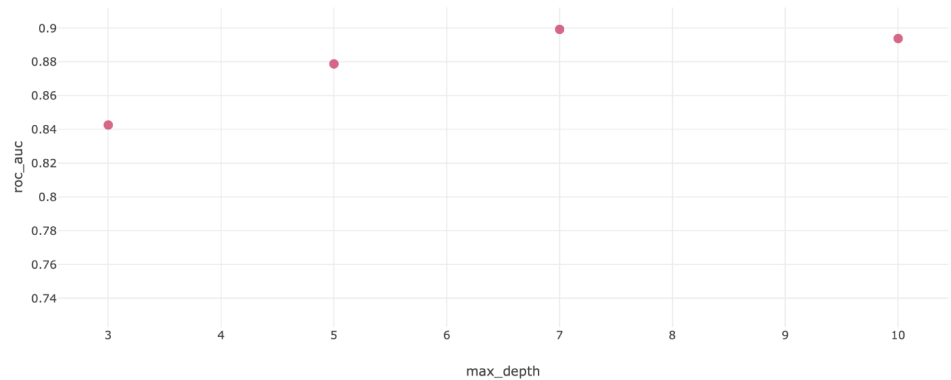
Parallel Coordinates Plot Scatter Plot Box Plot Contour Plot

X-axis:

max_depth

Y-axis:

roc_auc

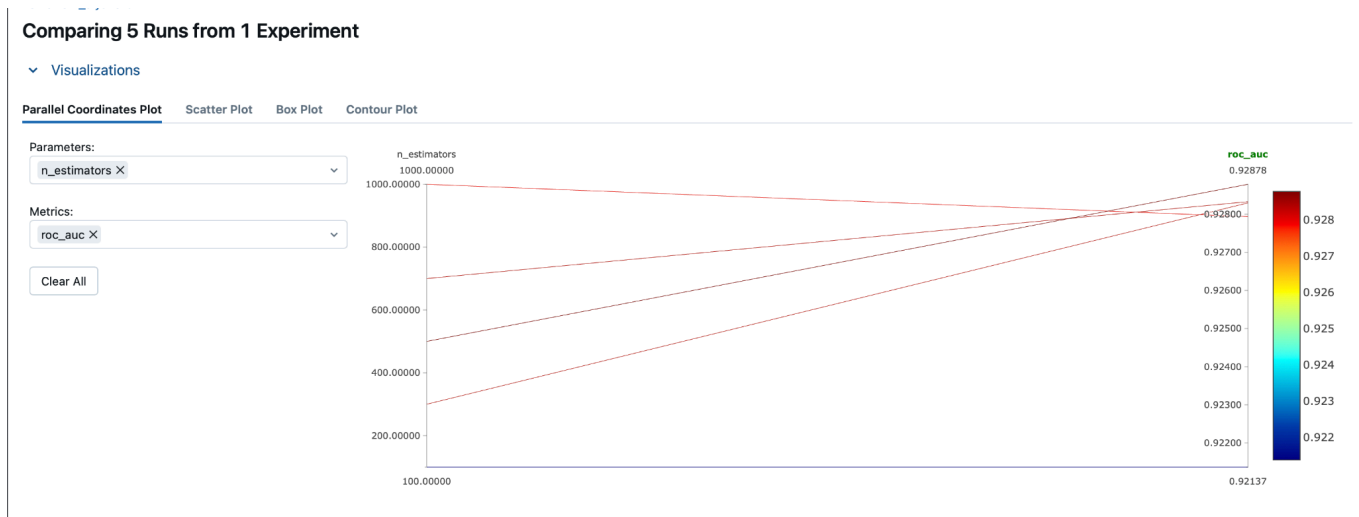


С увеличением глубины метрика растёт. Однако, если глубину совсем не ограничивать, то качество сильно падает, roc_auc = 0.736, сравнимо с линейной моделью. Зато наилучший вариант roc_auc = ~0.9, что практически эквивалентно случайному лесу. То есть одного дерева почти хватило, но все же лес чуть лучше) Возможно, если потюнуть еще и другие параметры, то можно получить эквивалент

- Эксперимент 4

Гипотеза: А куда без бустинга? Прогнозирую, бустинг будет лучше всего. А с увеличением количества оценщиков качество будет расти.

Оценим градиентный бустинг по сетке n_estimators = 100, 300, 500, 700, 1000.



Да, действительно получилось топ-качество roc_auc = 0.929, лучше чем все предыдущие запуски. Метрика была выбита на модели с n_estimators = 500. Дальше, видимо, пошло переобучение, т.к. N = 700 ниже, а N = 1000 еще ниже.

- Ссылка на лучший запуск по ROC-AUC

Название папки homework_ulyanova

run_id 360a2cd8392b4b9e8704faaf4b5b26ae

Название GradientBoosting_c2fd18c42675

<http://158.160.2.37:5000/#/experiments/4/runs/360a2cd8392b4b9e8704faaf4b5b26ae>

accuracy 0.8733749616132664

f1 0.7056864144658577

roc_auc 0.9287848901222169

precision 0.7960279119699409

recall 0.6337606837606837

average_precision 0.8284786929259411

- Дополнение

Хочу показать, что все виды артефактов корректно сохраняются в MLFlow

- csv с ошибками модели

GradientBoosting_fd8af12542c7

Register model

OverviewModel metricsSystem metricsTracesArtifacts

modelMLmodelconda.yamlmodel.pklpython_env.yamlrequirements.txtmodel_errors.csv

model_errors.csv115.75KB

Path: mlflow-artifacts:/4/27b4973964934516b9134dcf7310f91b/artifacts/model_errors.csv

Previewing the first 500 rows

	12	13	y_true	y_pred	y_proba
	4	0	0	1	0.7897067834531332
	4	0	1	0	0.3357196041384189
	4	0	0	1	0.6433358816611479
	4	0	1	0	0.4259179493450054
	4	0	0	1	0.7385485452890289

- PR-кривая

GradientBoosting_fd8af12542c7

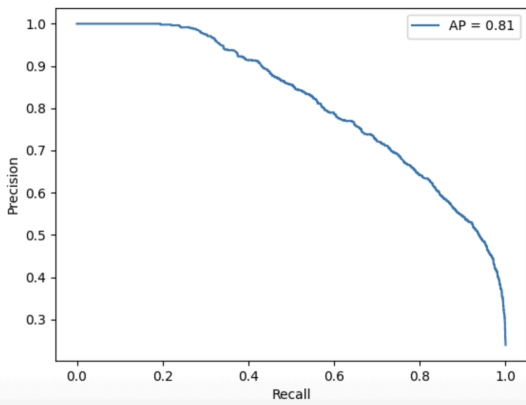
OverviewModel metricsSystem metricsTracesArtifacts

modelMLmodelconda.yamlmodel.pklpython_env.yamlrequirements.txtpr_curve.png

pr_curve.png20.96KB

Path: mlflow-artifacts:/4/40677167ceb946ae8dfea6d6a52ad2a8/artifacts/pr_curve.png

PR Curve



AP = 0.81

- confusion matrix

GradientBoosting_fd8af12542c7

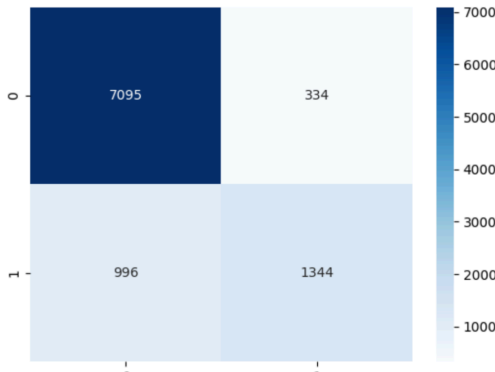
OverviewModel metricsSystem metricsTracesArtifacts

modelMLmodelconda.yamlmodel.pklpython_env.yamlrequirements.txtconfusion_matrix.png

confusion_matrix.png16.81KB

Path: mlflow-artifacts:/4/58deb7d415a74488ac071c62b79edff1/artifacts/confusion_matrix.png

Confusion Matrix



	0	1
0	7095	334
1	996	1344

- feature importances

GradientBoosting_fd8af12542c7

Overview Model metrics System metrics Traces **Artifacts**

▼ model

- MLmodel
- conda.yaml
- model.pkl
- python_env.yaml
- requirements.txt
- feature_importances.json**

feature_importances.json 521B

Path: mlflow-artifacts:/4/4999d49ac9cb472c903b781efa5f898f/artifacts/feature_importances.json

```
{
  "feature_0": 0.036103210630710404,
  "feature_1": 0.0639864795397712,
  "feature_2": 0.06089912290778886,
  "feature_3": 0.20791106700429418,
  "feature_4": 0.005124666521432742,
  "feature_5": 0.21812128135661238,
  "feature_6": 0.0007385106021379227,
  "feature_7": 0.0030337260674689246,
  "feature_8": 0.3386714659633919,
  "feature_9": 0.004216288462512151,
  "feature_10": 0.03664797855241153,
  "feature_11": 0.00015755097378003882,
  "feature_12": 0.023232051653905426,
  "feature_13": 0.0011565997637822236
}
```

- classification report

GradientBoosting_256e7da0f9ac

Overview Model metrics System metrics Traces **Artifacts**

▼ model

- MLmodel
- conda.yaml
- model.pkl
- python_env.yaml
- requirements.txt
- classification_report.json**

classification_report.json 626B

Path: mlflow-artifacts:/4/4633bf55968e4a0

```
{
  "0": {
    "precision": 0.8933655312658871,
    "recall": 0.9461569524835106,
    "f1-score": 0.9190037262208276,
    "support": 7429
  },
  "1": {
    "precision": 0.7895844292477643,
    "recall": 0.6414529914529915,
    "f1-score": 0.7078519217165763,
    "support": 2340
  },
  "accuracy": 0.8731702323676938,
  "macro avg": {
    "precision": 0.8414749802568258,
    "recall": 0.7938049719682511,
    "f1-score": 0.813427823968702,
    "support": 9769
  },
  "weighted avg": {
    "precision": 0.8685065100024612,
    "recall": 0.8731702323676938,
    "f1-score": 0.8684258551449808,
    "support": 9769
  }
}
```